



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 2 (10%) – Transaction Management

Name	:	MUHAMMAD SYAHMI FARIS BIN RUSLI & MUHAMMAD AFIQ DANIHS BIN MOHD HAZNI
Matric Id	:	A23CS0138 & A23CS0118
Section	:	02

Instructions:

This assignment evaluates your ability to apply the concept of serializability, differentiate between serial and non-serial transaction schedules and implement locking techniques to manage simultaneous transactions.

Academic Integrity

All students must work in a group of **TWO(2)**. The use of AI-generated tools or sharing of answers between students **is strictly prohibited** and will result in **zero marks**. You are required to complete the answers in **handwritten form**. Please take **clear screenshots** of your answers and submit them in **PDF format**.

QUESTION 1 (15 MARKS)

Consider the schedule transactions in Table 1 and answer all following questions (i) – (iv).

Table 1: Schedule T1, T2 and T3

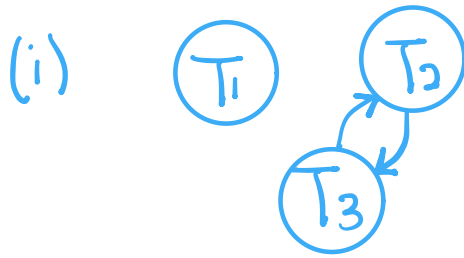
Time	T1	T2	T3
t ₁	begin transaction		
t ₂	read_item(X)		
t ₃	read_item(Y)	begin transaction	
t ₄	commit	read_item(M)	begin transaction
t ₅		read_item(N)	read_item(N)
t ₆		$M = (N + M) * 10$	read_item(M)
t ₇		write_item(M)	$N = N + 150$
t ₈		commit	write_item(N)
t ₉			commit

- i) Draw the precedence graph for the transaction schedule in Table 1 to check for serializability.
- ii) Explain the precedence graph produced in (i).
- iii) Apply the locking technique to the transaction schedule in Table 1, and revise the transaction schedule in Table 1 using the following format:

T1	T2	T3

- iv) Identify the problem(s) that arise in (iii) and then explain any potential problems that may still occur after applying the locking technique in concurrency control based on your answer in (iii).

Question 1



$\therefore$ Non conflict
Serializable

(ii) The presence of cycle $T_2 \rightarrow T_3$, $T_3 \rightarrow T_2$ shows that the concurrent schedule is not conflict serializable. The cycle implies a logical contradiction in execution order, which can lead to dirty read or lost update

(iii)

T_1	T_2	T_3
Begin S-lock(x) read-item(n) unlock(n) S-lock(m) read-item(n) unlock(y)	Begin n-lock(m) read-item(m) S-lock(N) wait	Begin n-lock(N) read-item(m) S-lock(m) wait

(iv) $\therefore$ There is a deadlock since a circular waiting exists in T_2 and T_3 . Besides, the starvation will arise because the transaction is perpetually delayed.

QUESTION 2 (15 MARKS)

Based on the transaction schedule in Table 2, answer all the following questions:

Table 2: Transaction Schedule for T4-T5-T6

↓ time	T4	T5	T6
	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
	$P = (P * 5) + Q$	read_item(Q)	
	write_item(P)	read_item(R)	begin transaction
	commit	$Q = Q + R$	read_item(R)
		write_item(Q)	$R = R - 100$
		commit	write_item(R)
			commit

- Draw the precedence graph for the transaction schedule in Table 2 to check for serializability.
- Explain the precedence graph produced in (i).
- Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

T4	T5	T6

QUESTION 3 (25 MARKS)

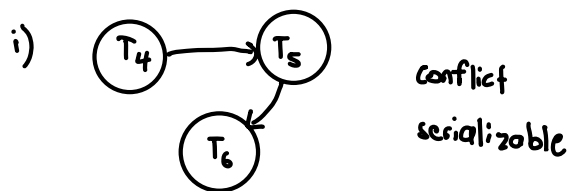
- Consider the schedule transactions in Table 3.1 and answer all following questions (i) – (iv).

Table 3.1: Schedule T7, T8, T9 and T10

Time	T7	T8	T9	T10
t ₁	begin transaction	begin transaction		
t ₂	read (X)	read (Y)	begin transaction	
t ₃	$X = X + 50$		read (X)	begin transaction
t ₄	write (X)			read (Z)
t ₅		read (X)	$X = X - 15$	$Z = Z + 20$
t ₆		$X = Y * 12$	write (X)	write (Z)
t ₇		write (X)	commit	
t ₈	commit	commit		commit

- Is the schedule in Table 3.1 a serial or non-serial schedule? State your reason.
- Draw the precedence graph for the transactions given in Table 3.1.
- Does the schedule in Table 3.1 have conflict-serializable or not? How can you determine this?
- What problem does the schedule in Table 3.1 face?

Question 2

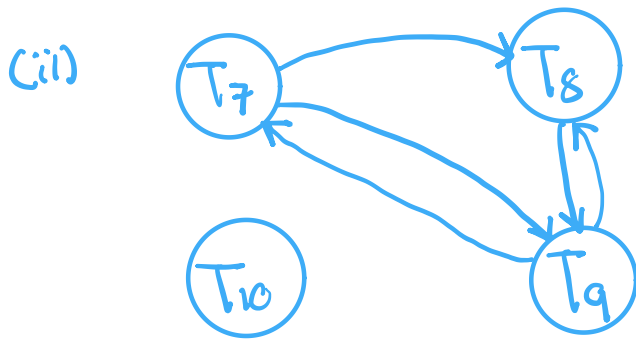


ii) The precedence graph is conflict serializable as it does not contain any loop

iii)	T_4	T_5	T_6
	<pre>begin x = lock(P) read-item(P) s-lock(Q) read-item(Q) P = (P * S) + Q write-item(P) unlock(P) & (Q)</pre>	<pre>begin s-lock(Q) read-item(Q) s-lock(R) read-item(R) WAIT x-lock(Q) Q = Q + R write-item(Q) unlock(Q) & (R)</pre>	<pre>begin s-lock(R) read-item(R) WAIT WAIT WAIT WAIT WAIT WAIT x-lock(R) R = R - 100 write-item(R) unlock(R)</pre>

Question 3a

(i) Non Serial. It allows multiple transaction to interleave before it completely end.



(iii) Non conflict serializable. The precedence graph isn't a cycle. It contains cycle at $T_7 \rightarrow T_9$, $T_9 \rightarrow T_7$, $T_8 \rightarrow T_9$ and $T_9 \rightarrow T_8$.

(iv) Lost updates. Multiple transaction concurrently read and write data item u.

- b) Consider the schedule transactions in Table 3.2 and answer all following questions (i) – (iv).

Table 3.2: Schedule T11, T12, T13 and T14

Time	T11	T12	T13	T14
t ₁	begin transaction			
t ₂	read (A)	begin transaction		
t ₃		read (B)	begin transaction	
t ₄	A = A + 200	B = B - 50	read (A)	begin transaction
t ₅	write (A)			read (C)
t ₆		write (B)	A = A - 30	C = C + 10
t ₇			write (A)	write (C)
t ₈	commit			
t ₉		read (A)		
t ₁₀		A = A * 2		
t ₁₁		write (A)		
t ₁₂		commit	commit	commit

- i) Implement the basic locking to the schedule in Table 3.2 by specifying the appropriate lock type (shared or exclusive) for each transaction.
- ii) Based on your answer in (i), state any other issues that may arise.
- iii) Implement the Two-Phase Locking (2PL) protocol to the schedule in Table 3.2.
- iv) Based on your answer in (iii), state any other issues that may arise.

Question 3b

(i)

T_1	T_2	T_3	T_4
begin n-lock(A) read(A) $A = A + 200$ write(A) unlock(A) commit	begin n-lock(B) read(B) $B = B - 50$ write(B) unlock(B) n-lock(A) wait read(A) $A = A * 2$ write(A) commit	begin n-lock(A) wait wait read(A) $A = A - 30$ unlock(A) Commit	begin n-lock(C) read(C) write(C) unlock(C) Commit

(ii) Starvation, Livelock, Deadlock

Question 3

b) iii)	T11	T12	T13	T14
	begin	begin	begin	begin
	$\pi_lock(A)$	$\pi_lock(B)$	$\pi_lock(A)$	$\pi_lock(C)$
	read(A)	read(B)	WAIT	read(C)
	$A = A + 200$	$B = B - 50$	⋮	$C = C + 10$
	write(A)	write(B)	⋮	write(C)
		$\pi_lock(B)$	Read(A)	
		WAIT	$A = A - 30$	
		⋮	write(A)	
			unlock(A)	
	unlock(A)	read(A)		unlock(C)
		$A = A * 2$		
		write(A)		
		unlock(A) & (B)		

iv) Cascading rollback

QUESTION 4 (15 MARKS)

Consider the schedule transactions in Table 4 and answer all following questions (i) – (v).

Table 4.1: Schedule T15, T16 and T17

Time	T15	T16	T17
t ₁	begin transaction		
t ₂	read (J)	begin transaction	
t ₃	J = J + 10	read (K)	begin transaction
t ₄	write (J)	K = K - 5	read (J)
t ₅	Read (K)	write (K)	J = J - 15
t ₆	K = K + 20		write (J)
t ₇	write (K)	commit	
t ₈	commit		commit

- Is the schedule in Table 4 a serial or non-serial schedule? State your reason.
- Draw the precedence graph for the transactions given in Table 4.
- Does the schedule in Table 4 have conflict-serializable or not? How can you determine this?
- What problem does the schedule in Table 4 face?
- Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

Submission

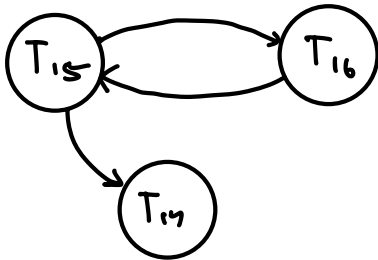
- Submission due date: **24 December 2025 (before 10:00PM)**.
- Each group must submit only **ONE** answer in PDF format.
- All work must be original and completed independently. The use of AI-generated tools or sharing of answers between students is strictly prohibited.
- Late submissions will be penalized according to faculty policy

Question 4

i) - Non-serial schedule

- Because it allows multiple transaction to be interleaved

ii)



iii) - Non conflict serializable

- Because there is a loop in the precedence graph

iv) - Last updates

- T16 writes K at t_5 and T15 writes again at t_1

v)	T15	T16	T17
	begin		
	$x_lock(J)$	begin	
	read(J)	$x_lock(K)$	
	$J = J + 10$	read(K)	begin
	write(J)	$K = K - 5$	$x_lock(J)$
	$x_lock(K)$	write(K)	WAIT
	WAIT	unlock(K)	:
	WAIT		
	read(K)		---
	$K = K + 20$		:
	write(K)		
	unlock(J) & (K)		---
			:
			read(J)
			$J = J - 15$
			write(J)
			unlock(J)